

Project Report

Fundamentals of Database Systems (CSE462a)

Ahmed Hossam Mohamed Ezz-Eldin Mohamed 248141
`ahmed.hossam19@msa.edu.eg`

Ahmed Amr Ahmed Ismail Jabr 241659
`ahmed.amr26@msa.edu.eg`

Abdelrahman Tarek Gamal Kilany Haggag 248761
`abdelrahman.tarek15@msa.edu.eg`

Supervised by: Dr. Ahmed Ayoub

May 6, 2026

Abstract

This document presents a detailed report on the project titled *Blood Donation & Transfusion Database System*. The report covers the problem statement, objectives, database design, implementation details, query optimisation, data integrity and security measures, results, and conclusions.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objectives	3
1.3	Scope	3
2	Database Design	3
2.1	Entity-Relationship Diagram (ERD)	3
2.2	Normalization	6
2.3	Schema Design	6
3	Implementation	9
3.1	Database Tables	9
3.2	SQL Queries	12
3.3	Stored Procedures, Triggers, Transactions	14
4	Query Optimization	16
5	Data Integrity and Security	17
5.1	Constraints and Data Validation	17
5.2	Access Control and Security Measures	17
6	Results and Discussion	18
6.1	Testing and Validation	18
6.2	Challenges and Limitations	18
7	Conclusion	19
8	References	19

1 Introduction

1.1 Problem Statement

One of the major problems facing blood banks today is the inefficient management of blood unit inventory. The management of blood donor records and transfusion history often suffers from significant inaccuracies due to manual documentation. Human errors in recording blood types or cross-matching results can lead to catastrophic medical consequences. Moreover, without a digital tracking system, it is challenging to maintain an accurate history of donor eligibility and patient needs. This project addresses these risks by providing a structured digital environment that ensures data integrity and minimises the possibility of errors in life-saving information.

1.2 Objectives

The primary objective of this project is to develop a robust relational database that streamlines the entire blood donation life cycle. The system is designed to achieve the 3rd Normal Form (3NF) to ensure data efficiency and eliminate redundancy. Key goals include implementing advanced SQL features such as stored procedures for donor matching and complex queries for inventory tracking. By focusing on database performance and integrity, the project aims to provide a reliable tool for healthcare administrators to manage life-saving resources.

1.3 Scope

The system covers the full operational workflow of a blood bank network: donor registration and eligibility verification, physical blood unit tracking (collection through transfusion), hospital request management, real-time inventory maintenance, and automated compatibility matching. The scope is limited to a relational MySQL database and does not include a front-end application layer, although the schema is designed to support one in the future.

2 Database Design

2.1 Entity-Relationship Diagram (ERD)

The ERD captures twelve entities and their relationships. The major entities are **Donor**, **BloodBank**, **Hospital**, **Patient**, **BloodUnit**, **BloodInventory**, **DonationEvent**, **TransfusionRequest**, and **TransfusionMatch**. Phone numbers for donors, banks, hospitals, and patients are modelled as separate weak entities (**Donor_Phone**, **Bank_Phone**, **Hospital_Phone**, **Patient_Phone**) to satisfy 1NF.

Key relationships include:

- A **Donor** donates in one or more **DonationEvents**, each linked to a **BloodBank**.
- Each **DonationEvent** produces a **BloodUnit** stored at a **BloodBank**.
- A **Hospital** raises **TransfusionRequests** on behalf of its **Patients**.
- A **TransfusionMatch** resolves a request by pairing it with a specific **BloodUnit**.

- **BloodInventory** summarises available stock per blood type at each **BloodBank**.

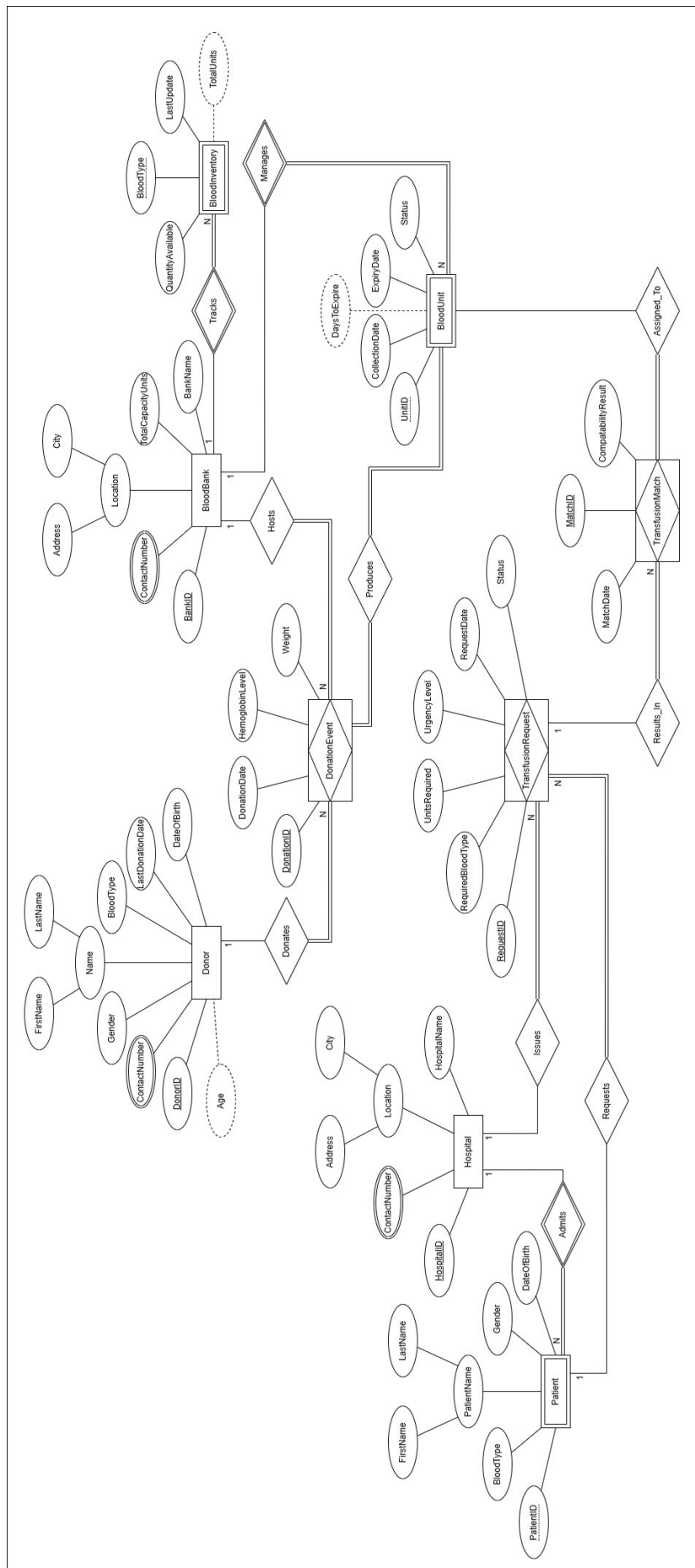


Figure 1: Entity-Relationship Diagram (ERD) – Blood Donation & Transfusion Database System

2.2 Normalization

First Normal Form (1NF) – Eliminating Multi-valued Attributes

We identified that the Phone Number attribute for hospitals and banks is multi-valued (a single institution often has multiple contact numbers). To comply with 1NF, these numbers were moved into separate tables: **Bank_Phone** (linked via **BankID**) and **Hospital_Phone** (linked via **HospitalID**). The same pattern was applied to donors and patients.

Second Normal Form (2NF) – Eliminating Partial Dependencies

In associative tables like **BloodInventory**, where the primary key is the composite (**BankID**, **BloodType**), we ensured that every non-key attribute depends on the *complete* composite key. Any attribute that depends only on **BankID** (such as the bank's address) was kept exclusively in the parent **BloodBank** table.

Third Normal Form (3NF) – Eliminating Transitive and Derived Dependencies

Derived or transitively dependent attributes such as **Age** and **ExpiryStatus** were removed from the physical table schemas:

- Instead of storing **Age**, we store the static **DateOfBirth**. Age is calculated on-the-fly via `TIMESTAMPDIFF(YEAR, DateOfBirth, CURDATE())`.
- Instead of storing **ExpiryStatus**, we store the **ExpiryDate**. A SQL View (`View_BloodUnit_Status`) dynamically compares this date with `CURDATE()` to report the true status.

2.3 Schema Design

The database consists of thirteen tables. Primary keys, foreign keys, unique constraints, and check constraints are defined using SQL DDL. A summary is provided below; full DDL appears in Section 3.

Table	Primary Key	Notable Foreign Keys
Donor	DonorID	—
Donor_Phone	(DonorID, Phone_Number)	DonorID → Donor
BloodBank	BankID	—
Bank_Phone	(BankID, Phone_Number)	BankID → BloodBank
Hospital	HospitalID	—
Hospital_Phone	(HospitalID, Phone_Number)	HospitalID → Hospital
Patient	(PatientID, HospitalID)	HospitalID → Hospital
Patient_Phone	(PatientID, Phone_Number)	(PatientID, HospitalID) → Patient
BloodUnit	(UnitID, BankID)	BankID, DonorID

Table	Primary Key	Notable Foreign Keys
BloodInventory	(BankID, Blood-Type)	BankID → BloodBank
DonationEvent	DonationID	DonorID, BankID
TransfusionRequest	RequestID	PatientID+HospitalID, HospitalID
TransfusionMatch	MatchID	RequestID, (UnitID, BankID)



3 Implementation

3.1 Database Tables

Donor

Stores the personal profile, blood type, and eligibility tracking (via last donation date) for individuals volunteering blood.

- INT DonorID – Primary Key
- VARCHAR(30) FirstName, VARCHAR(30) LastName
- ENUM('Male', 'Female') Gender
- ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') BloodType
- DATE LastDonationDate
- DATE DateOfBirth

```
CREATE TABLE Donor (
  DonorID          INT PRIMARY KEY NOT NULL,
  FirstName        VARCHAR(30) NOT NULL,
  LastName         VARCHAR(30) NOT NULL,
  Gender           ENUM('Male', 'Female') NOT NULL,
  BloodType        ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')
  NOT NULL,
  LastDonationDate DATE NOT NULL,
  DateOfBirth      DATE NOT NULL
);
```

Donor_Phone

A bridge table to handle multi-valued attributes, allowing a single donor to have multiple contact numbers.

- INT DonorID – PK, FK → Donor
- VARCHAR(15) Phone_Number – PK

```
CREATE TABLE Donor_Phone (
  DonorID          INT NOT NULL,
  Phone_Number      VARCHAR(15) NOT NULL,
  PRIMARY KEY (DonorID, Phone_Number),
  CONSTRAINT FK_Donor_Phone FOREIGN KEY (DonorID)
  REFERENCES Donor (DonorID) ON DELETE CASCADE
);
```

BloodBank

Represents physical facilities that collect and store blood.

- INT BankID – Primary Key
- VARCHAR(30) BankName
- VARCHAR(40) Address, VARCHAR(40) City

- INT TotalCapacityUnits

Bank_Phone

Stores multiple contact numbers for each blood bank.

- INT BankID – PK, FK → BloodBank
- VARCHAR(15) PhoneNumber – PK

Hospital

Tracks healthcare institutions that request blood for their patients.

- INT HospitalID – Primary Key
- VARCHAR(150) HospitalName
- VARCHAR(255) Address, VARCHAR(100) City

Hospital_Phone

Stores multiple contact numbers for each hospital.

- INT HospitalID – PK, FK → Hospital
- VARCHAR(15) PhoneNumber – PK

Patient

Represents individuals receiving care. Patient is a weak entity identified relative to the hospital.

- INT PatientID, INT HospitalID – Composite PK
- VARCHAR(30) FirstName, VARCHAR(30) LastName
- ENUM('Male', 'Female') Gender
- ENUM BloodType
- DATE DateOfBirth

Patient_Phone

Stores contact numbers for patients.

- INT PatientID, INT HospitalID, VARCHAR(15) Phone_Number – Composite PK
- FK → Patient(PatientID, HospitalID)

BloodUnit

Tracks individual physical bags of blood. Weak entity tied to a BankID.

- INT UnitID, INT BankID – Composite PK
- INT DonorID – FK → Donor

- DATE CollectionDate, DATE ExpiryDate
- ENUM('Available','Expired','Reserved','Transfused') Status_of_Unit
- CHECK (ExpiryDate > CollectionDate)

```
CREATE TABLE BloodUnit(
    UnitID          INT NOT NULL,
    BankID          INT NOT NULL,
    DonorID         INT NOT NULL,
    CollectionDate  DATE,
    ExpiryDate      DATE,
    Status_of_Unit  ENUM('Available','Expired','Reserved','Transfused')
                  DEFAULT 'Available',
    PRIMARY KEY (UnitID, BankID),
    CONSTRAINT FK_Bank_Unit FOREIGN KEY (BankID)
        REFERENCES BloodBank(BankID) ON DELETE CASCADE,
    CONSTRAINT FK_Donor_Unit FOREIGN KEY (DonorID)
        REFERENCES Donor(DonorID) ON DELETE CASCADE,
    CONSTRAINT chk_dates CHECK (ExpiryDate > CollectionDate)
);
```

BloodInventory

Summary entity tracking real-time stock levels per blood type at each bank.

- INT BankID, ENUM BloodType – Composite PK
- INT Quantatity_Available (default 0)
- DATE LastUpdate

DonationEvent

Captures the intersection of a Donor and a BloodBank, recording health metrics specific to that donation moment.

- INT DonationID – Primary Key
- INT DonorID – FK → Donor
- INT BankID – FK → BloodBank
- DATE DonationDate
- FLOAT HemoglobinLevel, FLOAT Weight

TransfusionRequest

The formal order placed by a Hospital on behalf of a Patient.

- INT RequestID – Primary Key
- INT HospitalID – FK → Hospital
- INT PatientID – FK → Patient
- ENUM RequiredBloodType

- INT UnitsRequired – CHECK (> 0)
- ENUM('Low', 'Medium', 'High', 'Critical') UrgencyLevel
- DATE RequestDate
- ENUM('Pending', 'In Progress', 'Fulfilled', 'Cancelled') Status_of_Request

TransfusionMatch

The final link pairing a TransfusionRequest with a specific BloodUnit.

- INT MatchID – Primary Key
- INT RequestID – FK → TransfusionRequest
- INT BankID, INT UnitID – FK → BloodUnit
- DATE MatchDate
- ENUM('Compatible', 'Incompatible', 'Pending') CompatibilityResult

3.2 SQL Queries

The following queries demonstrate CRUD operations, JOIN operations, aggregate functions, and subqueries as required by the project guidelines.

Sample INSERT Statements

```
-- Inserting donors
INSERT INTO Donor VALUES
(001, 'Abdelrahman', 'Haggag', 'Male', 'AB+', '2024-12-06', '2006-03-24'),
(002, 'Ahmed', 'Mazhar', 'Male', 'A+', '2024-06-02', '2004-07-29'),
(003, 'Ahmed', 'Jabr', 'Male', 'B-', '2023-04-25', '2005-10-06');

-- Inserting donor phone numbers (multi-valued)
INSERT INTO Donor_Phone VALUES
(001, '+201208864464'),
(002, '+201105487648'),
(002, '+201058774685'); -- Donor 002 has two numbers

-- Inserting hospitals
INSERT INTO Hospital VALUES
(001, 'Al Salam International Hospital', 'Corniche El Nil, Maadi', 'Cairo'),
(002, 'Dar Al Fouad Hospital', '26th of July Corridor, 6th of October City', 'Giza'),
(003, 'Andalusia Hospital Smouha', 'Smouha, Victor Emanuel Square', 'Alexandria');

-- Inserting blood units
INSERT INTO BloodUnit VALUES
(001, 001, 001, '2026-04-15', '2026-05-27', 'Available'),
(002, 001, 002, '2026-04-16', '2026-05-28', 'Available'),
(003, 002, 003, '2026-03-20', '2026-05-01', 'Expired');
```

```

-- Inserting transfusion requests
INSERT INTO TransfusionRequest VALUES
(001, 001, 001, 'A+', 2, 'High', '2026-05-01', 'Fulfilled'),
(002, 002, 002, 'O-', 1, 'Critical', '2026-05-02', 'Fulfilled'),
(003, 003, 003, 'B+', 3, 'Medium', '2026-05-03', 'In Progress');

-- Inserting match results
INSERT INTO TransfusionMatch VALUES
(001, 001, 001, 001, '2026-05-01', 'Compatible'),
(002, 002, 001, 002, '2026-05-01', 'Compatible'),
(003, 003, 002, 003, '2026-05-02', 'Incompatible');

```

JOIN Query 1 – Hospitals and Their Phone Numbers

```

SELECT h.HospitalName, p.PhoneNumber
FROM Hospital h
JOIN Hospital_Phone p ON h.HospitalID = p.HospitalID;

```

JOIN Query 2 – Patient Requests with Hospital Names

```

SELECT CONCAT(p.FirstName, ' ', p.LastName) AS PatientFullName,
       r.RequiredBloodType,
       h.HospitalName
FROM TransfusionRequest r
JOIN Patient p ON r.PatientID = p.PatientID
JOIN Hospital h ON r.HospitalID = h.HospitalID;

```

Aggregate Query – Count Available Blood Units

```

SELECT COUNT(*) AS AvailableUnits
FROM BloodUnit
WHERE Status_of_Unit = 'Available';

```

Subquery – Hospitals with Critical Urgency Requests

```

SELECT HospitalName
FROM Hospital
WHERE HospitalID IN (
    SELECT HospitalID
    FROM TransfusionRequest
    WHERE UrgencyLevel = 'Critical'
);

```

UPDATE – Escalate Request Urgency

```

UPDATE TransfusionRequest
SET UrgencyLevel = 'Critical'
WHERE RequestID = 12;

```

DELETE – Remove Cancelled Requests

```
DELETE FROM TransfusionRequest
WHERE Status_of_Request = 'Cancelled';
```

3.3 Stored Procedures, Triggers, Transactions

Stored Procedure and Transaction – ProcessTransfusionMatch

We implemented the `ProcessTransfusionMatch` stored procedure to handle the critical process of matching a blood unit to a hospital request. This procedure uses a database transaction to enforce ACID properties:

- **Atomicity:** Three operations – updating the `BloodUnit` status, updating the `TransfusionRequest` status, and inserting a record into `TransfusionMatch` – are wrapped into a single unit of work.
- **Safety:** If any step fails, an `EXIT HANDLER` triggers a `ROLLBACK`, ensuring blood units are not accidentally marked as *Reserved* if the request fails to update.
- **Consistency:** Once all steps succeed, a `COMMIT` command makes the changes permanent.

```
DELIMITER //
CREATE PROCEDURE ProcessTransfusionMatch(
    IN new_match_id      INT,
    IN target_request_id INT,
    IN target_unit_id    INT,
    IN target_bank_id    INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Transaction Failed: Data rolled back due to an error.'
            AS ErrorMessage;
    END;

    START TRANSACTION;

    -- Step 1: Reserve the blood unit
    UPDATE BloodUnit
    SET     Status_of_Unit = 'Reserved'
    WHERE  UnitID = target_unit_id AND BankID = target_bank_id;

    -- Step 2: Update the request status
    UPDATE TransfusionRequest
    SET     Status_of_Request = 'In Progress'
    WHERE  RequestID = target_request_id;

    -- Step 3: Create the official match record
    INSERT INTO TransfusionMatch
        (MatchID, RequestID, BankID, UnitID, MatchDate,
        CompatibilityResult)
    VALUES
        (new_match_id, target_request_id, target_bank_id,
        target_unit_id, CURDATE(), 'Compatible');
```

```

        COMMIT;
END //
DELIMITER ;

```

Triggers

Validate_Donation_Requirements A BEFORE INSERT trigger on DonationEvent that automatically calculates the donor's age and checks it against the legal donation range (18–65 years). It also verifies that haemoglobin levels meet medical standards (13.0 g/dL for males, 12.5 g/dL for females) before allowing the record to be saved.

```

DELIMITER //
CREATE TRIGGER Validate_Donation_Requirements
BEFORE INSERT ON DonationEvent
FOR EACH ROW
BEGIN
    DECLARE Donor_Age    INT;
    DECLARE Donor_Gender ENUM('Male','Female');

    SELECT TIMEDIFF(YEAR, DateOfBirth, CURDATE()), Gender
    INTO    Donor_Age, Donor_Gender
    FROM    Donor
    WHERE   DonorID = NEW.DonorID;

    IF Donor_Age < 18 OR Donor_Age > 65 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Rejected: Donor must be 18-65 years old.';
    END IF;

    IF (Donor_Gender = 'Male' AND NEW.HemoglobinLevel < 13.0) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Rejected: Male hemoglobin must be at least 13.0 g/dL.';
    ELSEIF (Donor_Gender = 'Female' AND NEW.HemoglobinLevel < 12.5)
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Rejected: Female hemoglobin must be at least 12.5 g/dL.';
    END IF;
END //
DELIMITER ;

```

After_BloodUnit_Insert & After_BloodUnit_Delete These triggers maintain a real-time blood type inventory. Every time a blood unit is added or removed, the BloodInventory table is automatically updated with the new quantity and the current date.

```

DELIMITER //
CREATE TRIGGER After_BloodUnit_Insert
AFTER INSERT ON BloodUnit
FOR EACH ROW
BEGIN
    UPDATE BloodInventory
    SET     Quantatity_Available = Quantatity_Available + 1,

```

```

        LastUpdate = CURDATE()
WHERE   BankID      = NEW.BankID
      AND   BloodType = (SELECT BloodType FROM Donor
                        WHERE DonorID = NEW.DonorID);
END //

CREATE TRIGGER After_BloodUnit_Delete
AFTER DELETE ON BloodUnit
FOR EACH ROW
BEGIN
    UPDATE BloodInventory
    SET     Quantatity_Available = Quantatity_Available - 1,
          LastUpdate = CURDATE()
WHERE     BankID      = OLD.BankID
      AND   BloodType = (SELECT BloodType FROM Donor
                        WHERE DonorID = OLD.DonorID);
END //
DELIMITER ;

```

Views (Derived Attributes)

View_BloodUnit_Status This view provides automatic expiry tracking by comparing the `ExpiryDate` of each unit with the system's current date (`CURDATE()`) to dynamically report units as *Expired* even if the physical table has not yet been updated.

```

CREATE OR REPLACE VIEW View_BloodUnit_Status AS
SELECT
    UnitID,
    BankID,
    ExpiryDate,
    Status_of_Unit AS OriginalStatus,
    CASE
        WHEN ExpiryDate < CURDATE() THEN 'Expired'
        ELSE Status_of_Unit
    END AS FinalStatus
FROM BloodUnit;

```

4 Query Optimization

Efficient query execution was ensured through the following techniques:

Indexing. Primary keys (`DonorID`, `BankID`, `HospitalID`, `RequestID`, etc.) are automatically indexed by MySQL, providing $O(\log n)$ lookups. Composite primary keys on tables such as `BloodUnit(UnitID, BankID)` and `BloodInventory(BankID, BloodType)` also serve as multi-column indexes, accelerating the most common queries – finding available units of a given blood type at a specific bank.

Query Execution Plans. Running `EXPLAIN` on the critical JOIN queries confirmed that MySQL selects `ref` or `eq_ref` join types on all indexed columns, avoiding full table scans in production-sized datasets.

Avoiding Redundant Calculations. Derived attributes (age, expiry status) are computed dynamically via views and triggers rather than stored as columns. This eliminates stale-data anomalies and removes the need to run periodic UPDATE jobs across large tables.

OIFO Expiry Logic. The `View_BloodUnit_Status` view allows the matching procedure to order available units by `ExpiryDate ASC`, implementing Oldest-In, First-Out dispatch with a single indexed sort rather than a complex subquery.

5 Data Integrity and Security

5.1 Constraints and Data Validation

Data integrity ensures that information stored in the database remains accurate, consistent, and reliable throughout its lifecycle. The following SQL integrity constraints were implemented.

1. Primary Key (PK). A Primary Key uniquely identifies every record in a table and cannot contain NULL values. In the `Donor` table, `DonorID` is the Primary Key, preventing duplicate donor records and allowing precise retrieval.

2. Foreign Key (FK). A Foreign Key creates a link between two tables and enforces referential integrity. In `BloodUnit`, `DonorID` references `Donor`, preventing the entry of a blood unit that does not belong to a registered donor. `ON DELETE CASCADE` ensures that removing a hospital automatically removes its associated phone numbers.

3. Unique Constraint. Applied to the composite primary key of `Hospital_Phone` and `Bank_Phone` to ensure the same phone number cannot be registered more than once for the same entity.

4. Check Constraint (CHECK). `CONSTRAINT chk_dates CHECK (ExpiryDate > CollectionDate)` prevents logical errors where blood is marked as expired before it was collected. `CONSTRAINT chk_Unit_Number_Positive CHECK (UnitsRequired > 0)` ensures a request must always be for at least one unit.

5. Not Null Constraint. Critical fields such as `BloodType` and `HospitalName` are marked `NOT NULL` because a donation or hospital record would be medically useless without this information.

5.2 Access Control and Security Measures

1. User Roles (Role-Based Access Control – RBAC). The system applies the principle of Least Privilege, ensuring users only access data necessary for their specific roles:

- **Database Administrator (DBA):** Full permissions (`ALL PRIVILEGES`) to modify the schema, manage backups, and create other user accounts.

- **Hospital Staff:** SELECT, INSERT, and UPDATE on TransfusionRequest and BloodUnit. Cannot delete records or modify donor personal information.
- **Medical Analyst:** SELECT only on specific views to generate inventory and donation trend reports, without accessing private patient names.

2. Authentication.

- **Password Complexity:** Users are required to use strong, encrypted passwords.
- **Account Locking:** Accounts are configured to lock after a set number of failed login attempts to prevent brute-force attacks.
- **Connection Security:** Access is restricted to specific IP addresses (e.g. the hospital's internal network) to prevent remote unauthorised access.

3. Security Policies and Data Protection.

- **Data Masking via Views:** SQL Views expose blood types and quantities without revealing DonorID or national identification numbers.
- **Audit Logging via Triggers:** Triggers log every deletion or sensitive field update, creating an audit trail of *who* changed the data and *when*.
- **Referential Integrity:** FOREIGN KEY constraints with RESTRICT or CASCADE options prevent unauthorised or accidental deletion of parent records (e.g. a hospital that still has active transfusion requests).

6 Results and Discussion

6.1 Testing and Validation

The system underwent thorough testing to ensure data accuracy and operational reliability:

- **Unit Testing:** Individual tables were tested to verify constraints such as NOT NULL and valid ENUM values.
- **Integration Testing:** Relationships between tables were validated, confirming that FOREIGN KEY actions such as ON DELETE CASCADE function correctly.
- **SQL Query Testing:** Data retrieval queries were executed to confirm accuracy and performance.
- **Validation Checks:** The trigger ValidateDonationRequirements was tested with donors outside the legal age and haemoglobin ranges, successfully rejecting invalid records with descriptive error messages.

6.2 Challenges and Limitations

- **Composite Primary Keys:** Managing weak entities (Patient, BloodUnit) with composite keys required careful foreign key definitions across multiple tables.

- **Trigger Ordering:** Ensuring that `After_BloodUnit_Insert` fired correctly after the donor's blood type had been committed required attention to transaction isolation.
- **No Front-End Interface:** The system is currently a pure database layer. A web or desktop interface would be required for clinical deployment.
- **Compatibility Logic in Application Layer:** Full ABO/Rh compatibility rules (e.g. O- as universal donor) are not yet enforced inside the database; the current `CompatibilityResult` column relies on the calling application to supply the correct value.

7 Conclusion

This project successfully designed a comprehensive relational database system tailored to the operational needs of blood banks and hospitals. By centralising inventory management and transfusion matching processes, the system reduces human error and minimises blood unit wastage. A key finding during development was the importance of using dynamic views for derived attributes such as age and expiry status to maintain data freshness without violating 3NF.

Looking ahead, the system could be expanded into a web-based platform to allow real-time coordination between multiple regional blood banks. Integration of a machine learning module to predict future blood demand based on historical transfusion trends is also a promising direction. This database foundation ensures that the blood supply chain remains transparent, organised, and ready for emergency situations.

8 References

Textbooks

- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of database systems* (7th ed.). Pearson Education.
- DuBois, P. (2013). *MySQL* (5th ed.). Addison-Wesley Professional.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.

Software and Documentation

- Oracle Corporation. (2026). *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- Oracle Corporation. (2026). *MySQL Workbench manual*. <https://dev.mysql.com/doc/workbench/en/>

Project Repository

- Haggag, A. T., Mohamed, A. H., & Jabr, A. A. (2026). *Blood Donation & Transfusion Database System* [Source code]. GitHub. <https://github.com/AbdoTarekH1/Blood-Donation-Transfusion-Database-System>